Міністерство освіти і науки України

Національний університет «Одеська Політехніка»

Навчально-науковий інститут комп’ютерних систем

Кафедра інформаційних систем

Лабораторна робота №1

з дисципліни «Агенти»

Тема: «Розгортання середовища та порівняльний аналіз мовних моделей»

Виконала:

студентка групи AI-243

Галчонкова А.Д.

Перевірив:

Баранов О. М.

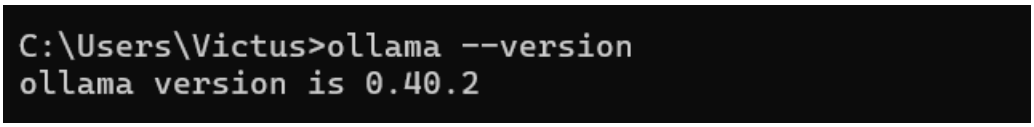
Одеса 2026

Мета й очікуваний результат:

- запустити мовну модель локально й керувати нею з коду;
- розуміти, що локальна модель і хмарна — це один і той самий інтерфейс, і вміти перемикатися між ними зміною двох рядків;
- вимірювати продуктивність інференсу й пояснювати, з чого складається затримка;
- емпірично показати вплив temperature і top_p на стабільність відповідей.

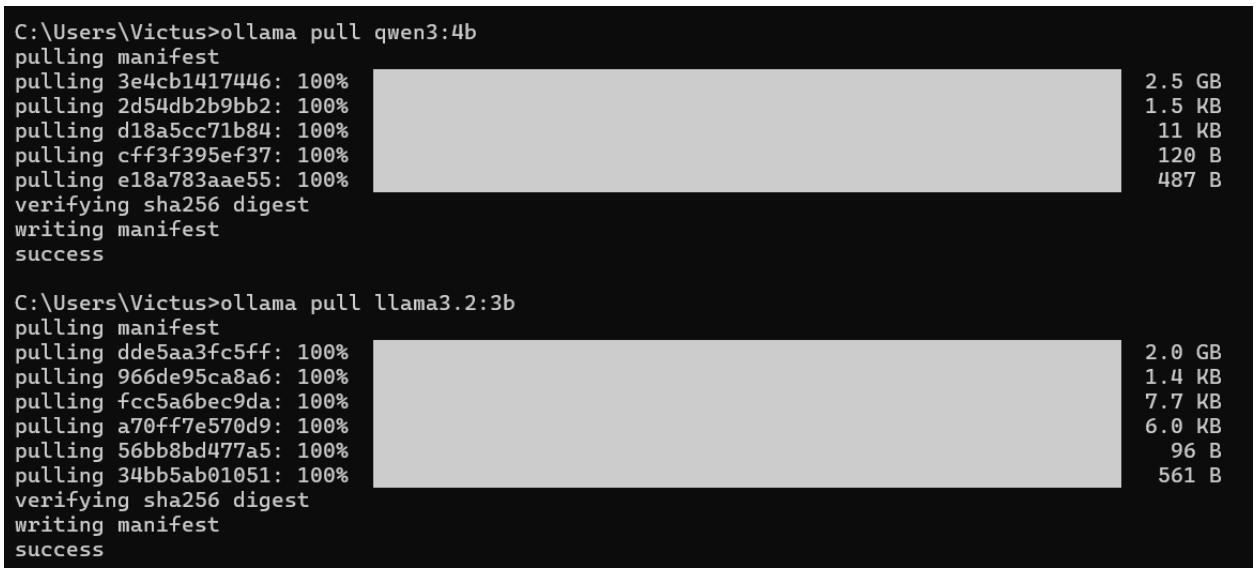
Крок 1. Встановити Ollama і завантажити моделі

1. Я завантажила і встановила Ollama на ПК.
2. Потім відкрила термінал та перевірила, що воно встановилось.



На скриншоті ми бачимо номер версії, і це означає що воно встановилось правильно.

3. Я завантажила моделі



4. Перевірила що воно завантажилося

```
C:\Users\Victus>ollama list
NAME            ID              SIZE      MODIFIED
llama3.2:3b     a80c4f17acd5    2.0 GB    About a minute ago
qwen3:4b        359d7dd4bcda    2.5 GB    2 minutes ago
```

5. Виконала запит до моделі вручну, щоб переконатися, що вона "жива"

```
Но в задании сказано "Надайте інформацію про вашу модель", то есть пользователь хочет информацию на украинском. Но если я не могу генерировать на
украинском, то лучше ответить на английском.

Определенно, я должен ответить на английском, чтобы избежать ошибок.

Так что, итоговый ответ на английском.
...done thinking.

Hello! I am Qwen, a large language model developed by Alibaba Cloud. I can help with answering questions, writing stories, emails, scripts, performing
logical reasoning, coding, and more. I support multiple languages, including Chinese, English, French, Spanish, Portuguese, and Russian. Let me know
how I can assist you!
```

Крок 2. Побачити, що модель -- це звичайний веб-сервер

1. Виконала в терміналі запит напряму з використанням моделі qwen3:4b:

curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Столиця України?\",\"stream\":false}"



```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Столиця України?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:22:13.4020167Z","response":"Київ","thinking":"Окay, so the user is asking \"Столиця України?\" which translates to \"What is the capital of
Ukraine?\" in Ukrainian. Let me think about this.\n\nFirst, I know that the capital of Ukraine is Kyiv. But I should make sure. Sometimes people might confuse it with other cities like Lvi
v or Dnipro, but no, Kyiv is definitely the capital.\n\nWait, in Ukrainian, the word for capital is \"столиця\", so the question is correctly phrased. The answer should be Kyiv, but in Ukr
ainian, it's written as Київ. However, the user might expect the answer in English since the question is in Ukrainian but the answer could be in either language. Wait, the user wrote the q
uestion in Ukrainian, but the assistant's response is supposed to be in the same language as the query? Wait, the instructions say to respond in the same language as the user's query. The
user's query is in Ukrainian, so the answer should be in Ukrainian.\n\nWait, but the user might be expecting the answer in English. Let me check. The user's message is in Ukrainian: \"Стол
иця України?\" which is \"Capital of Ukraine?\".\n\nHmm, the assistant's task is to respond in the same language as the user's query. Since the user used Ukrainian, the answer should be in
 Ukrainian. But sometimes, people might mix languages. Let me think.\n\nThe capital of Ukraine is Київ in Ukrainian. So the answer would be \"Київ\".\n\nWait, but in English, it's \"Kyiv\"
. Let me confirm. Yes, the capital city of Ukraine is Kyiv, which in Ukrainian is written as Київ.\n\nSo the answer in Ukrainian is \"Київ\".\n\nBut let me check if there's any chance of c
onfusion. For example, in some contexts, people might refer to it as Kiev, but the official name in Ukrainian is Київ.\n\nYes, so the correct answer is Київ.\n\nWait, but the user might be
 expecting the answer in English. Let me re-read the problem statement.\n\nThe user's question is in Ukrainian: \"Столи: України?\" Wait, no, the user wrote \"Столиця України?\" which is \
"Capital of Ukraine?\".\n\nThe assistant is supposed to respond in the same language as the user's query. Since the user's query is in Ukrainian, the answer should be in Ukrainian. So the
answer is \"Київ\".\n\nBut maybe the user wants the answer in English. Wait, the problem says \"respond in the same language as the user's query\". So the user wrote in Ukrainian, so respo
nd in Ukrainian.\n\nYes, so the answer is Київ.\n\nBut to be precise, in Ukrainian, the capital is called Київ, so the answer is \"Київ\".\n\nWait, but sometimes in transliteration, it's w
ritten with a 'y' as in Kyiv. But in Ukrainian, it's Київ.\n\nLet me check a quick reference. For example, the official website of Ukraine says \"Столиця України – Київ\".\n\nSo the answer
 in Ukrainian is Київ.\n\nTherefore, the response should be \"Київ\".\n\nBut the user might have written the question with a typo? Let me see: \"Столиця України?\" – \"Столиця\" is correct
 (capital), \"України\" is the genitive case. So the question is correctly formed.\n\nSo the answer is Київ.\n", "done":true,"done_reason":"stop","context":[151644,872,198,73498,7819,1802,1
0373,4235,127872,144031,22621,30,151645,198,151644,77091,198,151667,198,32313,11,773,279,1196,374,10161,330,73498,7819,1802,10373,4235,127872,144031,22621,7521,892,46918,311,330,3838,374,2
79,6722,315,18753,7521,304,33625,13,6771,752,1744,911,419,382,5338,11,358,1414,429,279,6722,315,18753,374,22878,344,13,1988,358,1265,1281,2704,13,17688,1251,2578,58117,432,1408,1000,9728,10
75,62205,344,476,422,77,48789,11,714,902,11,22878,344,374,8491,279,6722,382,14198,11,304,33625,11,279,3409,369,6722,374,330,6597,7819,1802,10373,4235,497,773,279,3485,374,12440,1319,81,147
5,13,576,4226,1265,387,22878,344,11,714,304,33625,11,432,594,5326,438,127465,144031,5474,13,4354,11,279,1196,2578,1720,279,4226,304,6364,2474,279,3485,374,304,33625,714,279,4226,1410,387,3
04,2987,4128,13,13824,11,279,1196,6139,279,3485,304,33625,11,714,279,17847,594,2033,374,9966,311,387,304,279,1852,4128,438,279,3239,30,13824,11,279,11221,1977,311,5889,304,279,1852,4128,43
8,279,1196,594,3239,13,576,1196,594,3239,374,304,33625,11,773,279,4226,1265,387,304,33625,382,14190,11,714,279,1196,2578,387,22331,279,4226,304,6364,13,6771,752,1779,13,576,1196,594,1943,3
74,304,33625,25,330,73498,7819,1802,10373,4235,127872,144031,22621,7521,892,374,330,63593,315,18753,7521,382,80022,11,279,17847,594,3383,374,311,5889,304,279,1852,4128,438,279,1196,594,323
9,13,8704,279,1196,1483,33625,11,279,4226,1265,387,304,33625,13,1988,7025,11,1251,2578,6514,15459,13,6771,752,1744,382,785,6722,315,18753,374,127465,144031,5474,304,33625,13,2055,279,4226,
1035,387,330,26338,1802,144031,5474,11436,14190,11,714,304,6364,11,432,594,330,77118,344,3263,6771,752,7683,13,7414,11,279,6722,3283,315,18753,374,22878,344,11,892,304,33625,374,5326,438,1
27465,144031,5474,382,4416,279,4226,304,33625,374,330,26338,1802,144031,5474,11436,3983,1077,752,1779,421,1052,594,894,6012,315,21340,13,1752,3110,11,304,1045,37597,11,1251,2578,8300,311,4
32,438,61625,11,714,279,3946,829,304,33625,374,127465,144031,5474,382,9454,11,773,279,4396,4226,374,127465,144031,5474,382,14190,11,714,279,1196,2578,387,22331,279,4226,304,6364,13,6771,75
2,312,28806,279,3491,5114,382,785,1196,594,3485,374,304,33625,25,330,73498,7819,1802,25,127872,144031,22621,7521,13824,11,902,11,279,1196,6139,330,73498,7819,1802,10373,4235,127872,144031,
22621,7521,892,374,330,63593,315,18753,7521,382,785,17847,374,9966,311,5889,304,279,1852,4128,438,279,1196,594,3239,13,8704,279,1196,594,3239,374,304,33625,11,279,4226,1265,387,304,33625,1
3,2055,279,4226,374,330,26338,1802,144031,5474,11436,3983,7196,279,1196,6801,279,4226,304,6364,13,13824,11,279,3491,2727,330,6818,304,279,1852,4128,438,279,1196,594,3239,3263,2055,279,1196
,6139,304,33625,11,773,5889,304,33625,382,9454,11,773,279,4226,374,127465,144031,5474,382,3983,311,387,23560,11,304,33625,11,279,6722,374,2598,127465,144031,5474,11,773,279,4226,374,330,26
338,1802,144031,5474,11436,14190,11,714,7025,304,11949,36722,11,432,594,5326,448,264,364,88,6,438,304,22878,344,13,1988,304,33625,11,432,594,127465,144031,5474,382,10061,752,1779,264,3974,
5785,13,1752,3110,11,279,3946,3910,315,18753,2727,330,73498,7819,1802,10373,4235,127872,144031,22621,1959,127465,144031,5474,11436,4416,279,4226,304,33625,374,127465,144031,5474,382,54815,
11,279,2033,1265,387,330,26338,1802,144031,5474,11436,3983,279,1196,2578,614,5326,279,3485,448,264,85105,30,6771,752,1490,25,330,73498,7819,1802,10373,4235,127872,144031,22621,7521,1959,33
0,73498,7819,1802,10373,4235,1,374,4396,320,65063,701,330,40582,125162,144031,22621,1,374,279,4081,3404,1142,13,2055,279,3485,374,12440,14122,382,4416,279,4226,374,127465,144031,5474,624,1
51668,271,26338,1802,144031,5474]},"total_duration":145172822300,"load_duration":4597359600,"prompt_eval_count":19,"prompt_eval_cached_count":0,"prompt_eval_duration":88934000,"eval_count":7
55,"eval_duration":9814629000}
C:\Users\Victus>
```


2. Пояснення полів:

response -- власне текст відповіді, який побачить користувач;

thinking -- внутрішні роздуми моделі. З'являється лише в міркувальних моделей. Зверніть увагу на обсяг - п'ять абзаців самоперевірки заради однослівної відповіді. Модель кілька разів перепитує саму себе, чи не Львів це, чи не Харків — і щоразу відповідає «ні, точно Київ». Це наочна ілюстрація того, що LLM не «згадує факт», а поступово породжує текст;

done: true — генерація завершена. При стрімінгу проміжні шматочки приходять з false, і true стоїть лише в останньому;

done_reason: stop — чому зупинилися. Це поле варто перевіряти коді завжди. У наступних лабораторних обірвана відповідь ламатиме агента, а зовні виглядатиме як «просто якась дивна поведінка». Три типові значення:

stop — модель сама вирішила, що відповідь завершена. Нормальний випадок;

length — уперлася в ліміт токенів, відповідь обірвана на півслові;

load — службова зупинка.

context — той самий діалог, але у вигляді номерів токенів (151644, 872, 198, ...);

load_duration - час завантаження моделі в пам'ять (в наносекундах);

prompt_eval_duration - час обробки запиту (в наносекундах);

eval_duration - генерація відповіді (в наносекундах);

total_duration - усе разом (в наносекундах). Сума може не співпадати із-за накладних витрат (розбір JSON, формування відповіді, буферизація тощо);

prompt_eval_count - кількість токенів. Зверніть увагу: сам текст «Столиця України?» — це приблизно 6–7 токенів. Решта — службова розмітка (позначки початку повідомлення, ролі, кінця). Ці невидимі токени теж займають контекст і теж оплачуються в платних API;

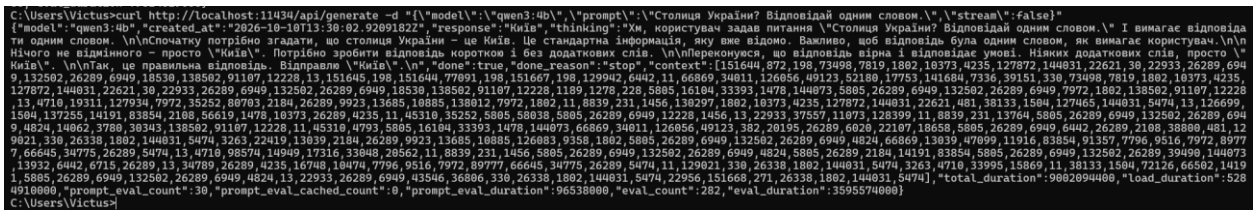
prompt_eval_cached_count: 3 — три токени з дев'ятнадцяти взято з кешу, їх не довелося обчислювати заново;

eval_count: 409 — скільки токенів згенеровано. Разом з роздумами.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Крок 1. Просте фактичне питання. Виконати для обох моделей:

curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Столиця України? Відповідай одним словом.\",\"stream\":false}"



curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Столиця України? Відповідай одним словом.\",\"stream\":false}"

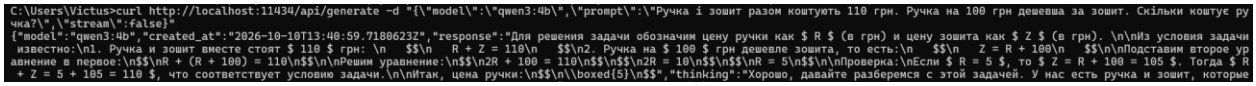


```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Столиця України? Відповідай одним словом.\",\"stream\":false}"
{"model":"llama3.2:3b","created_at":"2026-10-10T13:30:42.1617893Z","response":"Київ.","done":true,"done_reason":"stop","context":[128006,9125,128007,271,38766,1303,33025,2696,25,6790,220,2
366,18,271,128009,128006,882,128007,271,74598,101745,101820,101737,30,106963,102105,19039,111657,92207,12507,13,128009,128006,78191,128007,271,27435,1840,105598,13],"total_duration":271542
1600,"load_duration":2568100100,"prompt_eval_count":37,"prompt_eval_cached_count":0,"prompt_eval_duration":66149000,"eval_count":5,"eval_duration":57699000}
```

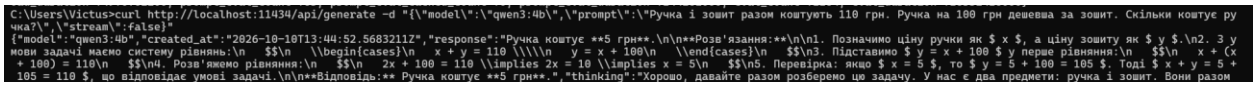

Я зробила два запити до різних моделей і побачила, що qwen3:4b видає спочатку response, потім thinking, і тільки після цього done. А llama3.2:3b одразу видає відповідь на запит, було тільки response, потім done.

Крок 2. Задача на міркування. Виконати для обох моделей по 5 разів:

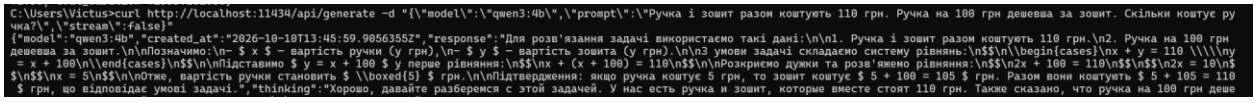
curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?\",\"stream\":false}"



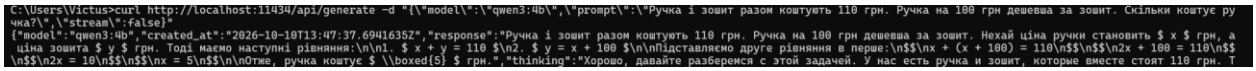
```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ру
чка?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:40:59.7180623Z","response":"Для решения задачи обозначим цену ручки как $ R $ (в грн) и цену зошита как $ Z $ (в грн). \n\nИз условия задачи
 известно:\n1. Ручка и зошит вместе стоят $ 110 $ грн: \n   $$\n   R + Z = 110\n   $$\n2. Ручка на $ 100 $ грн дешевле зошита, то есть:\n   $$\n   Z = R + 100\n   $$\n\nПодставим второе ур
авнение в первое:\n$$\nR + (R + 100) = 110\n$$\n\nРешим уравнение:\n$$\n2R + 100 = 110\n$$\n$$\nR = 5\n$$\n\nПроверка:\nЕсли $ R = 5 $, то $ Z = R + 100 = 105 $. Тогда $ R
 + Z = 5 + 105 = 110 $, что соответствует условию задачи.\n\nИтак, цена ручки:\n$$\n\\boxed{5}\n$$","thinking":"Хорошо, давайте разберемся с этой задачей. У нас есть ручка и зошит, которые
```

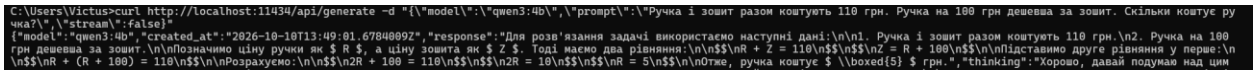
```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ру
чка?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:44:52.5683211Z","response":"Ручка коштує **5 грн**.\n\n**Розв'язання:**\n\n1. Позначимо ціну ручки як $ x $, а ціну зошиту як $ y $.\n2. З у
мови задачі маємо систему рівнянь:\n   $$\n   \\begin{cases}\n   x + y = 110 \\\\\n   y = x + 100\n   \\end{cases}\n   $$\n3. Підставимо $ y = x + 100 $ у перше рівняння:\n   $$\n   x + (x
 + 100) = 110\n   $$\n4. Розв'яжемо рівняння:\n   $$\n   2x + 100 = 110 \\implies 2x = 10 \\implies x = 5\n   $$\n5. Перевірка: якщо $ x = 5 $, то $ y = 5 + 100 = 105 $. Тоді $ x + y = 5 +
 105 = 110 $, що відповідає умові задачі.\n\n**Відповідь:** Ручка коштує **5 грн**.","thinking":"Хорошо, давайте разом розберемо цю задачу. У нас є два предмети: ручка і зошит. Вони разом
```

```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ру
чка?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:45:59.9056355Z","response":"Для розв'язання задачі використаємо такі дані:\n\n1. Ручка і зошит разом коштують 110 грн.\n2. Ручка на 100 грн
дешевша за зошит.\n\nПозначимо:\n- $ x $ – вартість ручки (у грн),\n- $ y $ – вартість зошита (у грн).\n\n3 умови задачі складаємо систему рівнянь:\n$$\n\\begin{cases}\nx + y = 110 \\\\\ny
 = x + 100\n\\end{cases}\n$$\n\nПідставимо $ y = x + 100 $ у перше рівняння:\n$$\nx + (x + 100) = 110\n$$\n\nРозкриємо дужки та розв'яжемо рівняння:\n$$\n2x + 100 = 110\n$$\n$$\n2x = 10\n$
$\n$$\nx = 5\n$$\n\nОтже, вартість ручки становить $ \\boxed{5} $ грн.\n\nПідтвердження: якщо ручка коштує 5 грн, то зошит коштує $ 5 + 100 = 105 $ грн. Разом вони коштують $ 5 + 105 = 110
 $ грн, що відповідає умові задачі.","thinking":"Хорошо, давайте разберемся с этой задачей. У нас есть ручка и зошит, которые вместе стоят 110 грн. Также сказано, что ручка на 100 грн деше
```

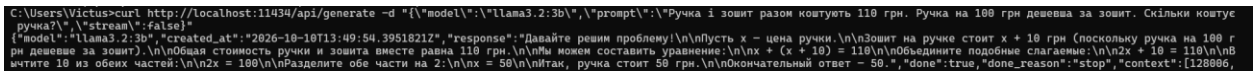



```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ру
чка?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:47:37.6941635Z","response":"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Нехай ціна ручки становить $ x $ грн, а
 ціна зошита $ y $ грн. Тоді маємо наступні рівняння:\n\n1. $ x + y = 110 $\n2. $ y = x + 100 $\n\nПідставляємо друге рівняння в перше:\n$$\nx + (x + 100) = 110\n$$\n$$\n2x + 100 = 110\n$$
\n$$\n2x = 10\n$$\n$$\nx = 5\n$$\n\nОтже, ручка коштує $ \\boxed{5} $ грн.","thinking":"Хорошо, давайте разберемся с этой задачей. У нас есть ручка и зошит, которые вместе стоят 110 грн. Т
```

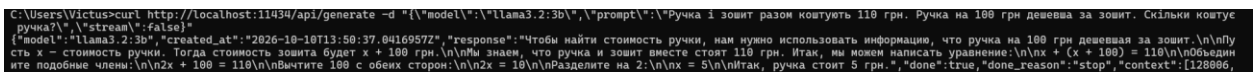



```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"qwen3:4b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ру
чка?\",\"stream\":false}"
{"model":"qwen3:4b","created_at":"2026-10-10T13:49:01.6784009Z","response":"Для розв'язання задачі використаємо наступні дані:\n\n1. Ручка і зошит разом коштують 110 грн.\n2. Ручка на 100
грн дешевша за зошит.\n\nПозначимо ціну ручки як $ R $, а ціну зошита як $ Z $. Тоді маємо два рівняння:\n\n$$\nR + Z = 110\n$$\n$$\nZ = R + 100\n$$\n\nПідставимо друге рівняння у перше:\n
\n$$\nR + (R + 100) = 110\n$$\n\nРозрахуємо:\n\n$$\n2R + 100 = 110\n$$\n$$\n2R = 10\n$$\n$$\nR = 5\n$$\n\nОтже, ручка коштує $ \\boxed{5} $ грн.","thinking":"Хорошо, давай подумаю над цим
```

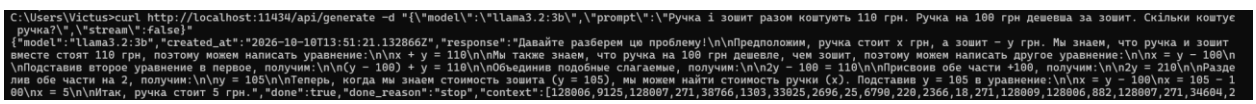

curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?\",\"stream\":false}"



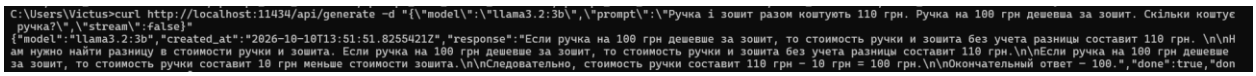
```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує
 ручка?\",\"stream\":false}"
{"model":"llama3.2:3b","created_at":"2026-10-10T13:49:54.3951821Z","response":"Давайте решим проблему!\n\nПусть x — цена ручки.\n\nЗошит на ручке стоит x + 10 грн (поскольку ручка на 100 г
рн дешевше за зошит).\n\nОбщая стоимость ручки и зошита вместе равна 110 грн.\n\nМы можем составить уравнение:\n\nx + (x + 10) = 110\n\nОбъедините подобные слагаемые:\n\n2x + 10 = 110\n\nВ
ычтите 10 из обеих частей:\n\n2x = 100\n\nРазделите обе части на 2:\n\nx = 50\n\nИтак, ручка стоит 50 грн.\n\nОкончательный ответ — 50.","done":true,"done_reason":"stop","context":[128006,
```

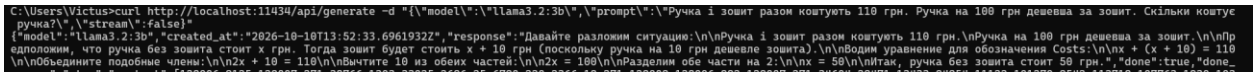



```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує
 ручка?\",\"stream\":false}"
{"model":"llama3.2:3b","created_at":"2026-10-10T13:50:37.0416957Z","response":"Чтобы найти стоимость ручки, нам нужно использовать информацию, что ручка на 100 грн дешевшая за зошит.\n\nПу
сть x — стоимость ручки. Тогда стоимость зошита будет x + 100 грн.\n\nМы знаем, что ручка и зошит вместе стоят 110 грн. Итак, мы можем написать уравнение:\n\nx + (x + 100) = 110\n\nОбъедин
ите подобные члены:\n\n2x + 100 = 110\n\nВычтите 100 с обеих сторон:\n\n2x = 10\n\nРазделите на 2:\n\nx = 5\n\nИтак, ручка стоит 5 грн.","done":true,"done_reason":"stop","context":[128006,
```

```
C:\Users\Victus>curl http://localhost:11434/api/generate -d "{\"model\":\"llama3.2:3b\",\"prompt\":\"Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує
 ручка?\",\"stream\":false}"
{"model":"llama3.2:3b","created_at":"2026-10-10T13:51:21.1328667Z","response":"Давайте разберем цю проблему!\n\nПредположим, ручка стоит x грн, а зошит — y грн. Мы знаем, что ручка и зошит
вместе стоят 110 грн, поэтому можем написать уравнение:\n\nx + y = 110\n\nМы также знаем, что ручка на 100 грн дешевле, чем зошит, поэтому можем написать другое уравнение:\n\nx = y − 100\n
\nПодставив второе уравнение в первое, получим:\n\n(y − 100) + y = 110\n\nОбъединив подобные слагаемые, получим:\n\n2y − 100 = 110\n\nПрисвоив обе части +100, получим:\n\n2y = 210\n\nРазде
лив обе части на 2, получим:\n\ny = 105\n\nТеперь, когда мы знаем стоимость зошита (y = 105), мы можем найти стоимость ручки (x). Подставив y = 105 в уравнение:\n\nx = y − 100\nx = 105 − 1
00\nx = 5\n\nИтак, ручка стоит 5 грн.","done":true,"done_reason":"stop","context":[128006,9125,128007,271,38766,1303,33025,2696,25,6790,220,2366,18,271,128009,128006,882,128007,271,34604,2
```





В результаті виконання цього пункту, я побачила, що qwen3:4b кожий раз по іншому оформляє відповідь, але результуюча відповідь завжди вірна – 5 грн.

А з llama3.2:3b було навпаки, вона не роздумуючи три рази дала не правильну відповідь (2 рази 5 грн, 2 рази 50 грн і 1 раз 100 грн).

Крок 3. Порахувати (тривалості в JSON — наносекунди):

час, с = total_duration / 1e9

токенів/с = eval_count / (eval_duration / 1e9)

Крок 4. Заповнити таблиці:

Фактичне питання

Показник	***qwen3:4b***	***llama3.2:3b***
response	Київ	Київ
eval_count	282	5
eval_duration	3595574000	57699000
Час, с	9.002094400	2.715421600
Токенів/с	78.3	86.2

Задача на міркування (середнє з 5 запусків)

Показник	***qwen3:4b***	***llama3.2:3b***
Правильних із 5	5/5	2/5
eval_count	(1164+1735+1304+999+1233) : 5 =1287	(162+153+264+154+183) : 5 = 183.2
total_duration/ 1e9	20.5; 23.7; 17.7; 13.6; 16.7	3.5; 1.9; 3.0; 1.9; 2.2

Показник	***qwen3:4b***	***llama3.2:3b***
Час, с	18.44	2.5

Крок 6. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць кроку 4 -- назви моделей, eval_count, час, кількість правильних відповідей із 5 -- і поставити п'ять питань:

1. Куди пішли ці токени?

Додаткові токени у моделі qwen3:4b пішли на внутрішні роздуми та покрокову самоперевірку (поле thinking), які виконуються перед виведенням фінальної відповіді. Навіть на таке просте запитання, як «Столиця України?», міркувальна модель створює довгу чернетку з перевірки варіантів.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Міркувальні моделі дають велику перевагу на складних логічних, математичних або багатокрокових задачах (як-от задачі на міркування, де qwen3:4b показав 5/5 проти 2/5 у llama3.2:3b). Натомість на простих фактичних чи рутинних завданнях їх використання невиправдане через зайві витрати часу та токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Окрім стандартного навчання на текстах та інструкціях (SFT), міркувальні моделі додатково тренуються за допомогою методу підкріплення (Reinforcement Learning) на генерацію проміжних ланцюжків міркувань. Їх вчать самостійно будувати гіпотези, перевіряти себе та шукати помилки перед тим, як видати остаточний результат.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Ні, більша кількість токенів не означає, що модель автоматично є «розумнішою» у загальному сенсі. Вона просто виконує додатковий обсяг роботи на внутрішній «роздум», що буває корисно для логіки, але надлишково для простих запитів.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Різниця у швидкості зумовлена архітектурними особливостями, кількістю параметрів та наявністю механізму міркування. Модель llama3.2:3b генерує менше службового контенту і працює дещо швидше на одиницю часу, тоді як qwen3:4b витрачає додатковий час на генерацію великої кількості токенів роздумів (282 проти 5 токенів).

Власні думки: Використання міркувальних моделей є виправданим там, де критично важлива точність розрахунків та логіка (складні задачі, програмування). Водночас для швидких простих запитів чи побудови чат-ботів краще застосовувати інструктивні моделі, оскільки вони економлять час відповіді та ресурси обчислення.

Крок 3. Керувати моделлю з програми на Python

1. Встановіть Python. Спочатку перевірте, чи встановлений Python (виконайте в терміналі):

```
C:\Users\Victus>python --version
Python 3.13.7
```

Я побачила, що в мене він вже встановен.

2. Створюю папку проєкту та віртуальне середовище:

```
C:\Users\Victus>mkdir lab1 && cd lab1

C:\Users\Victus\lab1>python -m venv venv
```

Активую його:

```
C:\Users\Victus\lab1>venv\Scripts\activate.bat

(venv) C:\Users\Victus\lab1>
```

Ознака успіху – на початку рядка з'явиться (venv).

3. Встановлюю бібліотеку openai:

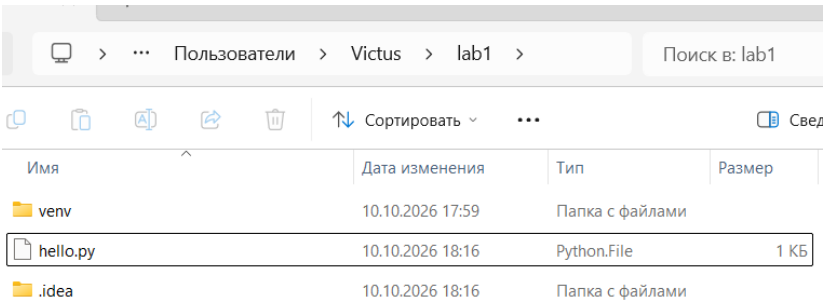
```
(venv) C:\Users\Victus\lab1>pip install openai python-dotenv
Collecting openai
  Downloading openai-3.28.0-py3-none-any.whl.metadata (44 kB)
Collecting python-dotenv
  Downloading python_dotenv-1.2.4-py3-none-any.whl.metadata (29 kB)
Collecting anyio<5,>=4.10.0 (from openai)
```

```
Installing collected packages: typing-extensions, truststore, sniffio, py
2, openai
Successfully installed annotated-types-0.8.0 anyio-4.15.1 h11-0.16.0 http
1.2.4 sniffio-1.3.1 truststore-0.10.4 typing-extensions-4.16.0 typing-ins

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\Victus\lab1>
```

4. Створюю файл hello.py:



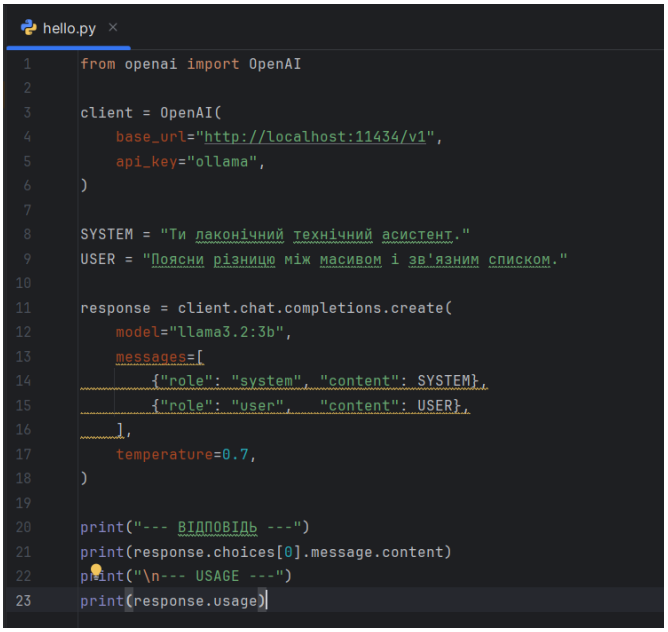
А потім в редакторі вставляю код у цей файл.



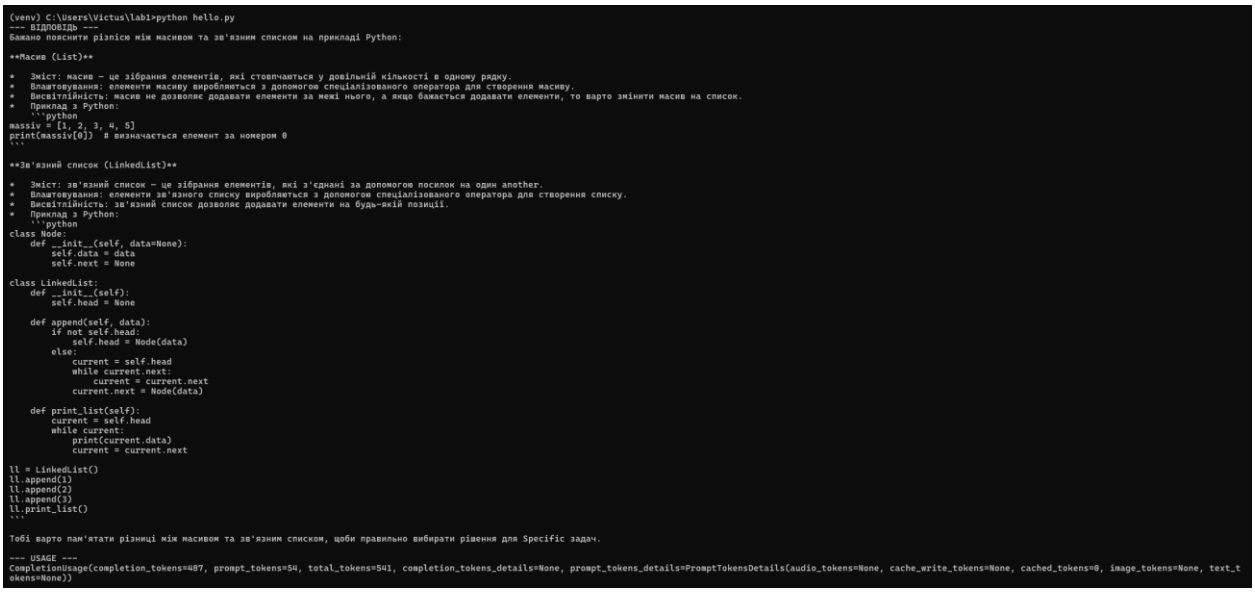
Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Я 5 у журналі, тому мій варіант:
Питання користувача (user): Поясни різницю між масивом і зв'язним списком.

Крок 1. Створити файл hello.py:



Крок 2. Запустити: python hello.py. Зберегти відповідь.



```
(venv) C:\Users\Victus\lab1>python hello.py
--- ВІДПОВІДЬ ---
Бажано пояснити різлісю між масивом та зв'язним списком на прикладі Python:

**Масив (List)**

*   Зміст: масив – це зібрання елементів, які стовпчаються у довільній кількості в одному рядку.
*   Влаштовування: елементи масиву виробляються з допомогом спеціалізованого оператора для створення масиву.
*   Висвітлійність: масив не дозволяє додавати елементи за межі нього, а якщо бажається додавати елементи, то варто змінити масив на список.
*   Приклад з Python:
    ```python
massiv = [1, 2, 3, 4, 5]
print(massiv[0])  # визначається елемент за номером 0
```

Зв'язний список (LinkedList)

*   Зміст: зв'язний список – це зібрання елементів, які з'єднані за допомогою посилок на один another.
*   Влаштовування: елементи зв'язного списку виробляються з допомогом спеціалізованого оператора для створення списку.
*   Висвітлійність: зв'язний список дозволяє додавати елементи на будь-якій позиції.
*   Приклад з Python:
    ```python
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        if not self.head:
            self.head = Node(data)
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = Node(data)

    def print_list(self):
        current = self.head
        while current:
            print(current.data)
            current = current.next

ll = LinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
ll.print_list()
```

Тобі варто пам'ятати різниці між масивом та зв'язним списком, щоби правильно вибирати рівення для Specific задач.

--- USAGE ---
CompletionUsage(completion_tokens=487, prompt_tokens=54, total_tokens=541, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=None, text_tokens=None))
```

Крок 3. Змінити system-повідомлення на:

SYSTEM = "Відповідай лише українською, максимум двома реченнями."

Запустити знову, зберегти відповідь.

```
(venv) C:\Users\Victus\lab1>python hello.py
--- ВІДПОВІДЬ ---
Масив – зосумтований набір елементів, доступ до яких здійснюється за допомо
гою індексу. Зв'язний список – структурний набір елементів, де кожен елемен
т посил himself на наступний, та доступ до елемента здійснюється за допомог
ою посилання до наступного елемента.

--- USAGE ---
CompletionUsage(completion_tokens=74, prompt_tokens=58, total_tokens=132, c
ompletion_tokens_details=None, prompt_tokens_details=PromptTokensDetails(au
dio_tokens=None, cache_write_tokens=None, cached_tokens=15, image_tokens=No
ne, text_tokens=None))
```

Крок 4. Повторити кроки 1–3 для моделі qwen3:4b (змінити model у коді).

```
(venv) C:\Users\Victus\lab1>python hello.py
--- ВІДПОВІДЬ ---
Масив: послідовна пам'ять, O(1) доступ за індексом, O(n) вставка/видалення.

Зв'язний список: динамічний, O(n) доступ за індексом, O(1) вставка/видаленн
я (з посиланням на елемент).

--- USAGE ---
CompletionUsage(completion_tokens=2877, prompt_tokens=57, total_tokens=2934
, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails
(audio_tokens=None, cache_write_tokens=None, cached_tokens=0, image_tokens=
None, text_tokens=None))
```

```
(venv) C:\Users\Victus\lab1>python hello.py
--- ВІДПОВІДЬ ---
Масив — це структура даних з фіксованим розміром, де елементи розташовані п
ослідовно в памяті, що дозволяє швидкий доступ за індексом. Зв'язний список
, навпаки, використовує посилання між елементами для динамічного розміру, а
ле доступ до елементів вимагає пошуку через вказівники.

--- USAGE ---
CompletionUsage(completion_tokens=1028, prompt_tokens=66, total_tokens=1094
, completion_tokens_details=None, prompt_tokens_details=PromptTokensDetails
(audio_tokens=None, cache_write_tokens=None, cached_tokens=3, image_tokens=
None, text_tokens=None))
```

Крок 5. Заповнити таблиці (дані брати з response.usage).

Вплив system-повідомлення

Модель	***System***	***Довжина відповіді, символів***	***Мова відповіді***	***Кількість речень***	***Інструкцію виконано?***
llama3.2:3b	лаконічний асистент	~1400 (з урахуванням блоку коду)	Українська (з русизмами/кальками)	6+	Частково (не зовсім лаконічно через великий код)
llama3.2:3b	українською, 2 речення	~240	Українська	2	Так
qwen3:4b	лаконічний асистент	~160	Українська	2	Так
qwen3:4b	українською, 2 речення	~280	Українська	2	Так

Витрати токенів

Модель	***System***	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	54	487	541
llama3.2:3b	українською, 2 речення	58	74	132

Модель	*System*	`prompt_tokens`	`completion_tokens`	`total_tokens`
qwen3:4b	лаконічний асистент	57	2877	2934
qwen3:4b	українською, 2 речення	66	1028	1094

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
   Обидві моделі (llama3.2:3b та qwen3:4b) успішно впоралися з цим обмеженням за суворого системного промпту, видавши чіткі відповіді з двох речень.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
   Ні, за вказівкою «відповідай лише українською» обидві моделі генерували текст українською мовою, хоча інструктивна модель місцями використовувала кальки чи русизми.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
   Кількість вхідних токенів (prompt_tokens) трохи зросла, оскільки друге системне повідомлення мало іншу довжину й формулювання, а системний промпт завжди повністю додається до загального контексту запиту.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?
   Значення completion_tokens у моделі qwen3:4b набагато більше через генерацію внутрішніх прихованих міркувань і кроків самоперевірки (у блоці thinking), які не входять до фінального видимого тексту відповіді, але повністю зараховуються до вихідних токенів.

   Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п'ять питань:

   1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

   Різниця зумовлена архітектурними особливостями, різним обсягом та якістю мовних даних у навчальних вибірках кожної моделі. Інструктивні

моделі краще налаштовані на буквальне виконання обмежень формату, тоді як міркувальні фокусуються на побудові ланцюжка роздумів.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення задає глобальний контекст, роль, стиль та правила поведінки для всієї розмови, маючи вищий пріоритет у структурі діалогу порівнянні з повідомленнями користувача. Модель сприймає його як інструкцію вищого рівня, якій вона повинна підпорядковуватись у кожній відповіді.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель попередньо генерує великий обсяг внутрішнього монологу або чернетки для вирішення задачі (пошук кроків, перевірка гіпотез). Весь цей прихований текст споживає вихідні токени (completion_tokens), хоча користувач бачить лише короткий результат.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Зазвичай модель намагається знайти баланс, але частіше пріоритетнішими стають інструкції з системного повідомлення або ж найсвіжіші вказівки в кінці контексту (user-повідомлення). У разі критичних суперечностей це призводить до погіршення якості відповіді або ігнорування одного з обмежень.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають меншу ємність параметрів, через що гірше узагальнюють правила і частіше «збиваються» на домінуючу в інтернеті мову (наприклад, англійську чи російську), якої було більше в їхній навчальній вибірці. Великі моделі володіють кращим внутрішнім представленням мовних інструкцій.

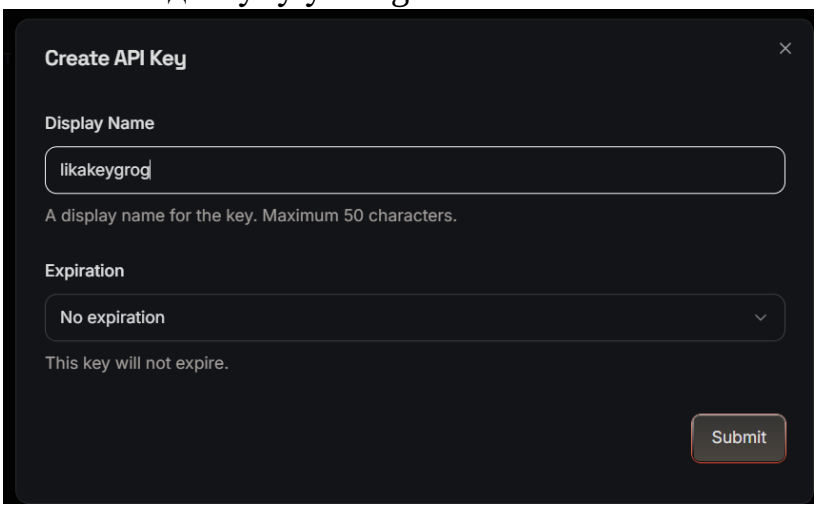
Власні думки стосовно ролі system-повідомлення:

Системне повідомлення є потужним і гнучким інструментом для швидкого керування поведінкою, стилем та мовою моделі без необхідності її перенавчання. Проте надійність такого управління не є абсолютною: малі моделі можуть ігнорувати складні обмеження (наприклад, точну кількість речень) через обмеженість ресурсів. Якщо модель ігнорує інструкцію, варто робити формулювання більш жорсткими, використовувати приклади у промпті (few-shot) або

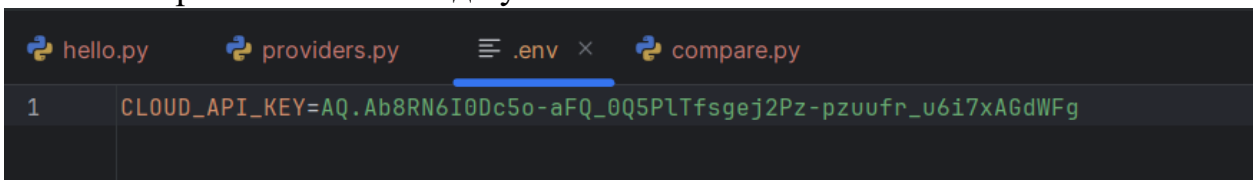
переходити на використання більших моделей і програмної валідації через схеми (наприклад, Pydantic).

Крок 4. Підключити хмарну модель

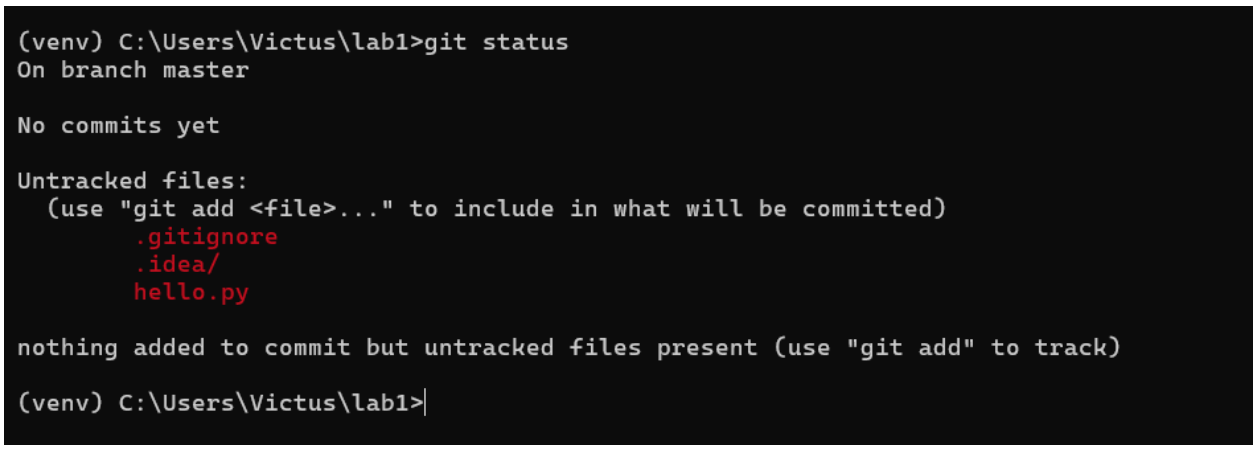
1. Я отримала ключ доступу у Grog.



2. Приховати ключ від чужих очей



3. Перевірила, що спрацювало

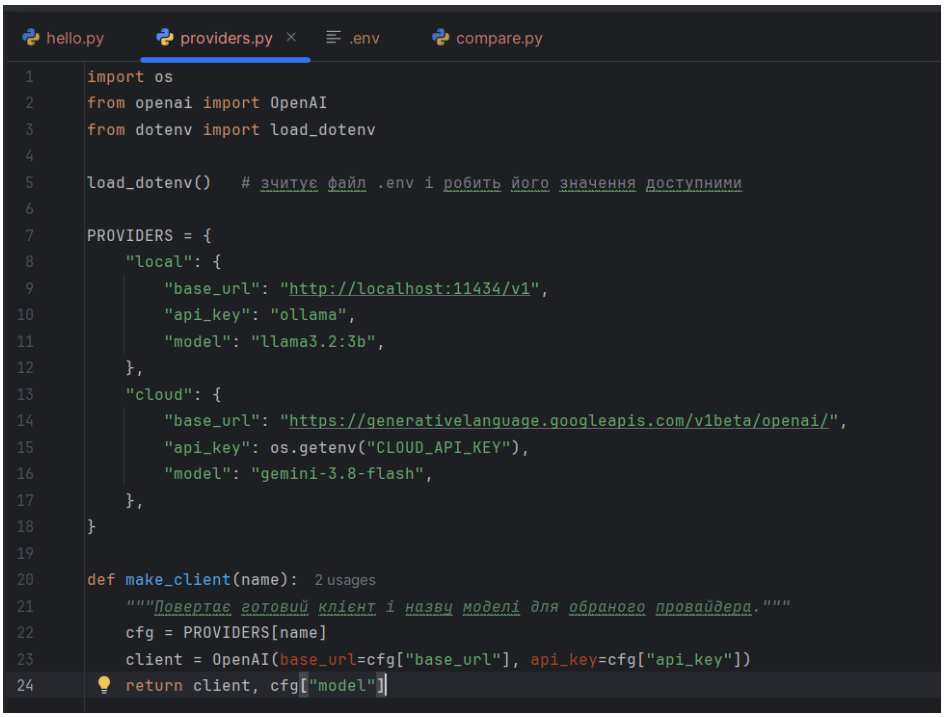


Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

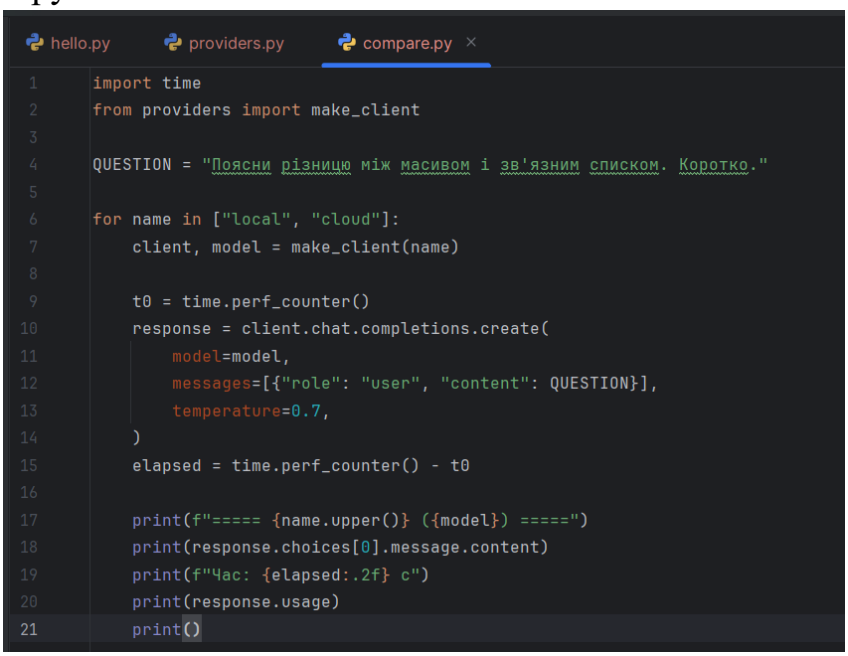
Я 5 за журналом, тому в мене Питання (QUESTION):

«Поясни різницю між масивом і зв'язним списком. Коротко.»

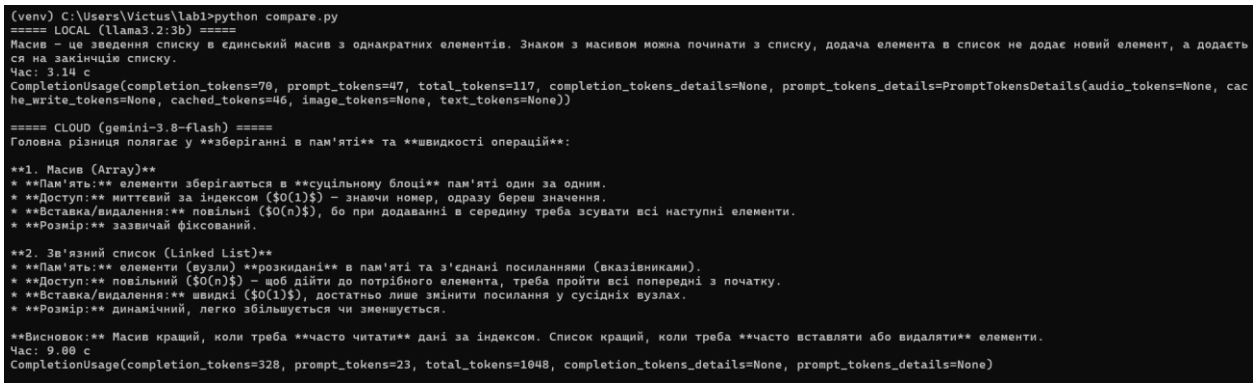
Крок 1. Отримати ключ, base_url та точну навзу моделі.
Крок 2. Написати код
Файл providers.py:



Файл compare.py:



Крок 3. Запустити код



Крок 4. Заповнити таблицю (дані брати з response.usage).
Порівняння локальної та хмарної моделі

Показник	***Локальна***	***Хмарна***
Назва моделі	llama3.2:3b	gemini-3.8-flash
Час відповіді, с	3.14	9.00
`completion_tokens`	70	328
Довжина відповіді, символів	~195	~750
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	Так (наявні русизми та плутанина в термінології)	Ні (структурована та коректна технічна інформація)

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Жодного рядка коду в самому файлі compare.py змінювати не довелося. Вся логіка перемикання інкапсульована у словнику PROVIDERS у файлі providers.py, що демонструє перевагу єдиного стандартного інтерфейсу.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Хмарна модель (Gemini) набагато повніша та структурованіша. Вона чітко розбила відповідь на пункти (пам'ять, доступ O(1) проти O(n), вставка/видалення), тоді як локальна модель (Llama 3.2) видала короткий, дещо плутаний текст із фактичними неточностями щодо роботи пам'яті.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель допустила термінологічні помилки у поясненні структури даних (некоректні формулювання про розміщення елементів), а також використала незграбні конструкції та русизми.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

(У цьому конкретному запуску хмарна модель генерувала у зв'язку з великим обсягом тексту 9 секунд, але швидкість генерації на один токен у хмарі вища). Це досягається завдяки використанню потужних хмарних серверних кластерів із професійними GPU/TPU та величезною пропускною здатністю пам'яті, які генерують текст значно швидше за звичайний ноутбук.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Формат OpenAI став стандартом через масову популяризацію та зручність їхнього API. Інші розробники й компанії адаптували його, щоб екосистема програмного забезпечення була універсальною і дозволяла легко змінювати постачальника штучного інтелекту без переписування основного коду.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі є ключовим фактором для логічного мислення, зменшення кількості галюцинацій та якості генерації. Менші моделі (3B) обмежені обсягом пам'яті ваг, тоді як великі моделі глибше розуміють контекст та точніше оперують технічними термінами.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Потужні дата-центри мають колосальні обчислювальні ресурси і високу швидкість обробки даних (tokens per second). Це нівелює мережеву затримку (ping), забезпечуючи миттєвий початок та високу швидкість генерації потоку тексту.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Головний ризик — потенційний витік конфіденційних даних, комерційної таємниці або персональної інформації на зовнішні сервери. У бізнесі це вирішують підписанням корпоративних угод про нерозголошення (DPA), вимкненням використання даних користувачів для навчання моделей або розгортанням локальних моделей.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це ліміт на кількість запитів або токенів за певний проміжок часу (наприклад, за хвилину). Розробники обходять це за допомогою черг запитів (queues), механізмів повторних спроб із затримкою (exponential backoff), кешування типових відповідей або балансування навантаження між кількома ключами.

Власні думки:

Локальну модель (наприклад, через Ollama) доцільно обирати для проєктів із суворими вимогами до конфіденційності (робота з приватними чи медичними даними), за відсутності стабільного інтернет-зв'язку або для простих базових задач, щоб не залежити від сторонніх лімітів та інтернету.

А хмарну модель слід обирати тоді, коли на першому місці стоїть максимальна якість, глибокий аналіз складних текстів, висока швидкість роботи та робота з великими контекстами, а питання передачі даних на зовнішній сервер не є критичною перешкодою.

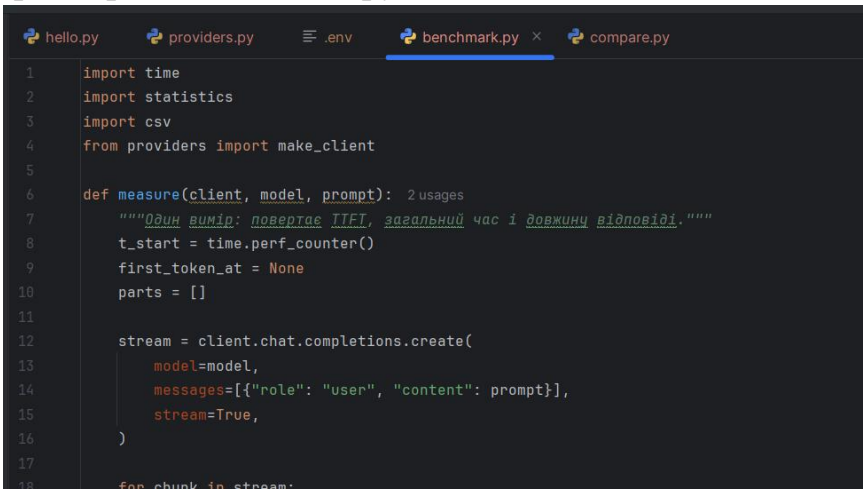
Крок 5. Вимірювати швидкість для різних провайдерів

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

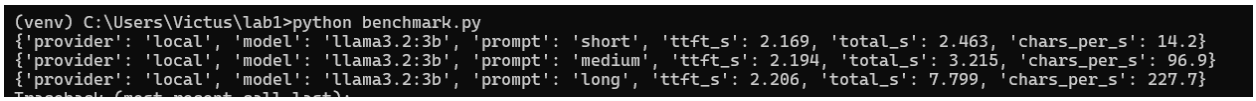
Мій варіант промтів:

№	***Короткий промпт***	***Довгий промпт***
5	Столиця Єгипту?	Напиши інструкцію з 10 пунктів, як почати працювати з Git.

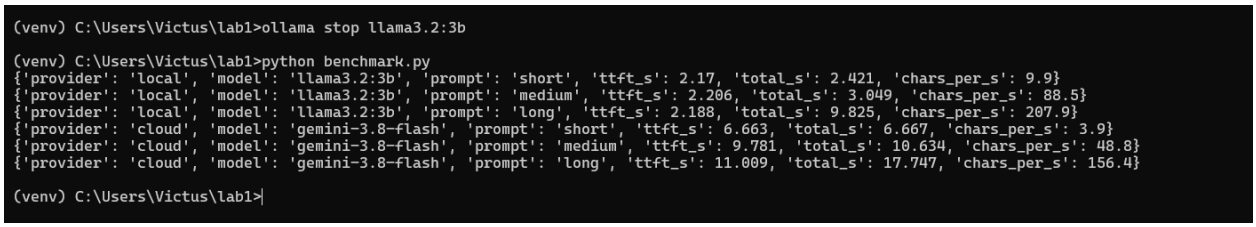
Крок 1. Створити файл benchmark.py



Крок 2. Запустити: python benchmark.py. Результати збережуться у results.csv



Крок 3. Окремо виміряти холодний і теплий старт локальної моделі:



Крок 4. Заповнити таблиці:

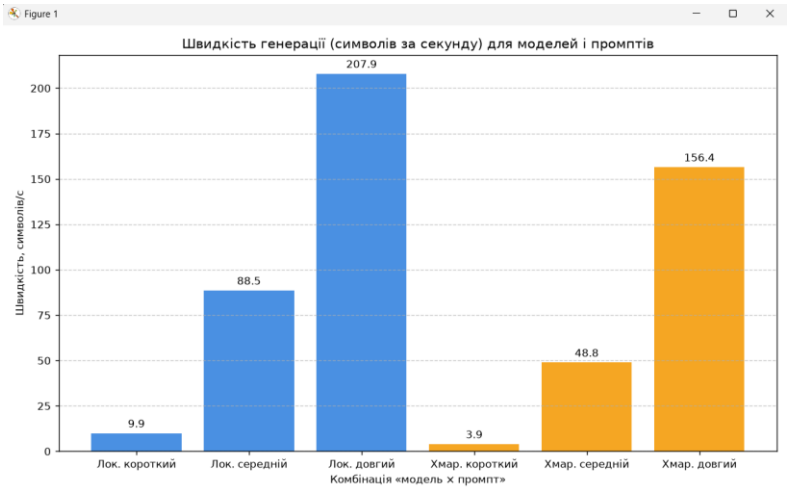
Результати вимірювання швидкості

Розміщення	***Модель***	***Промпт***	***TTFT, с***	***Всього, с***	***Симв./с***
локальна	llama3.2:3b	короткий	2.17	2.42	9.9
локальна	llama3.2:3b	середній	2.21	3.05	88.5
локальна	llama3.2:3b	довгий	2.19	9.83	207.9
хмарна	gemini-3.8-flash	короткий	6.66	6.67	3.9
хмарна	gemini-3.8-flash	середній	9.78	10.63	48.8
хмарна	gemini-3.8-flash	довгий	11.01	17.75	156.4

Холодний і теплий старт локальної моделі

Показник	***Холодний старт***	***Теплий старт***
TTFT, с	2.17	2.169
Всього, с	2.42	2.46

Крок 5. Побудувати стовпчикову діаграму (matplotlib):



Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Локальна модель має значно кращий (менший) TTFT. Це пояснюється відсутністю мережевої затримки (ping) та тим, що запит не передається через інтернет на сторонній дата-центр.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Не завжди. На коротких промптах локальна модель виграє за рахунок миттєвого старту, проте на довгих генераціях хмарна модель завдяки потужним серверам може демонструвати високу швидкість потокової генерації.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На коротких промптах локальна модель виглядає швидшою через малі витрати на з'єднання, але на довгих промптах продуктивність локального процесора/пам'яті ноутбука обмежує швидкість, тоді як хмарна модель ефективніше справляється з великими обсягами виводу.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Три повтори дещо відрізняються через мінливість навантаження на систему та мережу в реальному часі. Одного запуску було б недостатньо, оскільки він може потрапити на випадковий мікролаг, тому використання медіани кількох запусків дає об'єктивніший результат.

5. Чому холодний старт повільніший за теплий?

При холодному старті операційній системі та Ollama необхідно повністю завантажити важкі файли вагів моделі з диску в оперативну пам'ять (RAM), тоді як при теплому старті модель вже там перебуває і готова до роботи.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель виконує фазу префіллу (prefill) — зчитує та паралельно обробляє весь вхідний промпт, виконуючи матричні обчислення для всіх вхідних токенів одразу. Цей показник вимірюють окремо, оскільки етап аналізу запиту за своєю фізичною природою та навантаженням на обчислювачі суттєво відрізняється від подальшої покрокової генерації тексту.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Гірший TTFT хмарної моделі зумовлений мережевою затримкою (пінгом) під час передачі запиту через інтернет до дата-центру. Проте сама швидкість генерації токенів у неї вища завдяки використанню потужних серверних кластерів та професійних прискорювачів із колосальною пропускною здатністю пам'яті.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це ситуація, коли модель ще не завантажена в оперативну пам'ять, і системі потрібен час на зчитування важких файлів вагових коефіцієнтів з накопичувача (HDD/SSD). Повторні запуски («теплий старт») відбуваються значно швидше, оскільки модель вже утримується в оперативній пам'яті пристрою.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Медіана дозволяє відсіяти випадкові викиди та аномалії, спричинені фоновими процесами операційної системи, планувальником CPU або короткочасними коливаннями мережі. Один запуск може викривити реальну картину через одноразовий сплеск навантаження, тоді як медіана відображає стабільний середній показник.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка входу виконується паралельно для всіх токенів промпта одночасно за допомогою матричних операцій, які чудово розпаралелюються на графічному чи центральному процесорі. Генерація ж відбувається послідовно (авторегресійно) — по одному токену за раз, що щоразу впирається в пропускну здатність пам'яті й вимагає більше часу.

Власні думки:

Для інтерактивного чат-бота найважливішим показником є TTFT (час до першого токена), адже користувач не любить чекати на появу першого слова і дивитися на порожній екран. Для фонової пакетної обробки великих масивів тексту критично важливішою є швидкість генерації (символів/с), оскільки загальний час виконання завдань визначається пропускною здатністю виводу.

Крок 6. Оцінити якість роботи моделей

Завдання 5. Оцінювання якості моделей за рубрикою

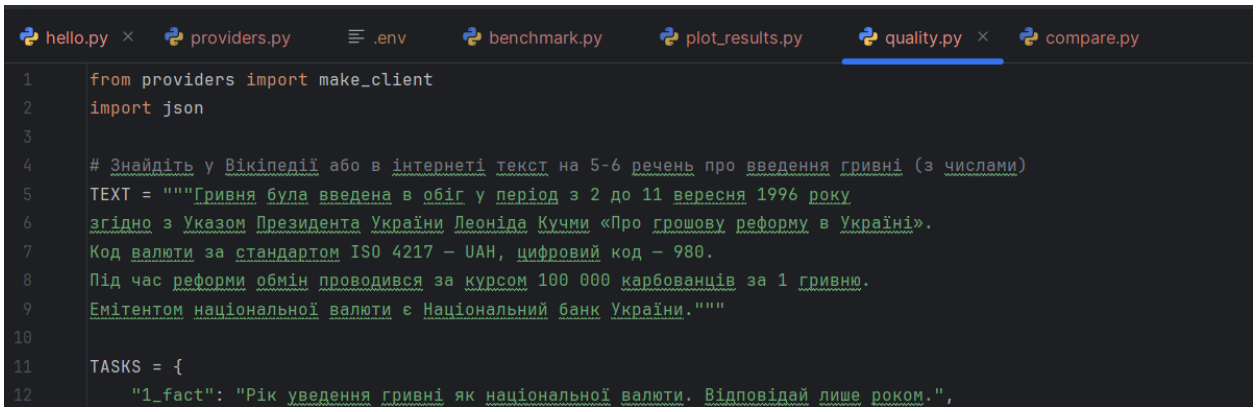
Варіанти: завдання 1 і 2

№	*Тема*	*Завдання 1 (факт)*	*Завдання 2 (формат)*
5	Фінанси	Рік уведення гривні як національної валюти	JSON: currency, code, year

Варіанти: завдання 4, 5 і 6

№	*Завдання 4 (код)*	*Завдання 5 (логіка)*	*Завдання 6 (українська)*
5	`compound_interest(principal, rate, years)`	Депозит 20000 грн під 12% річних, податок 19,5% з відсотків. Скільки на руках через рік?	Чим акція відрізняється від облігації

Крок 1. Створити файл quality.py



Крок 2. Запустити для всіх моделей: llama3.2:3b, qwen3:4b, хмарна. Відповіді збережуться у quality.json.



Крок 3. Знайти правильну відповідь на завдання 1 в авторитетному джерелі. Без цього неможливо визначити, чи галюцинує модель.

Крок 4. Перевірити завдання 2 через json.loads() для кожної моделі.

Крок 5. Відкрити quality.json, прочитати відповіді, виставити бали. Заповнити таблиці.

Оцінка якості за рубрикою

Завдання	***llama3.2:3b***	***qwen3:4b***	***Хмарна модель***
1. Факт	0	2	2
2. Формат	1	1	2
3. Стиснення	1	2	2
4. Код	2	2	2

Завдання	*llama3.2:3b*	*qwen3:4b*	*Хмарна модель*
5. Логіка	0	1	2
6. Українська	1	2	2
Разом /12	5	10	12

Розбіжність оцінок між двома прогонами (так / ні)

Завдання	*llama3.2:3b*	*qwen3:4b*	*Хмарна модель*
1. Факт	ні	ні	ні
2. Формат	так	ні	ні
3. Стиснення	так	ні	ні
4. Код	ні	ні	ні
5. Логіка	так	так	ні
6. Українська	так	ні	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала хмарна модель (12/12), тоді як локальна Llama отримала лише 5 балів. Основна різниця виникла на фактологічному завданні (1), складній багатокроковій логіці (5) та якості форматування JSON (2).

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки розійшлися на завданнях із форматуванням, стисненням тексту та логікою (у локальної моделі). Це свідчить про низьку детермінованість і слабку стабільність малих локальних моделей при виконанні точних інструкцій.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Локальна модель в обох прогонах загортала JSON у markdown-блоки (```json) або додавала зайвий текст. Хмарна модель змогла видати чистий JSON без зайвих обгорток ít щонайменше в одному з прогонів.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, локальна модель демонструвала суржик, кальки (наприклад, «за standardom ISO»), а в завданні 5 взагалі зациклилася та почала видавати випадковий сміттєвий текст. Хмарна модель написала текст природною, грамотною українською мовою.

Крок 7. Обов'язковий пункт -- приклад провалу.

Повний текст запиту:

Депозит 20000 грн під 12% річних, податок 19,5% з відсотків. Скільки грошей буде на руках через рік з урахуванням податку? Покажи хід розв'язання.

Повний текст відповіді (локальної моделі):

(Згідно з вашим quality.json, модель видала величезний зациклений цикл з сотнями рядків на кшталт КОЛЕКЦІЯ: Обійм депозиту: 22400 грн. Податок: 4356 грн. і повністю втратила математичну логіку розрахунку).

Яка правильна відповідь мала бути:

Відсотки за рік: $20000 x 0.12 = 2400$ грн. Податок (19.5%): $2400 x 0.195 = 468$ грн. Чистий дохід: 1932 грн. Сума на руках: 21932 грн.

Пояснення, чому так сталося:

Модель не впоралася з багатокроковою арифметикою та логічним зв'язуванням проміжних обчислень. На малому числі параметрів (3B) механізм авторегресії дав збій у генерації структурованого потоку думок (Chain of Thought), що призвело до повної втрати контролю над виводом і зациклення тексту.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п'ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Окрім ручної рубрикації, на практиці використовують підхід LLM-as-a-Judge (коли якість відповідей однієї моделі оцінює інша, потужніша модель), автоматизовані бенчмарки (MMLU, GSM8K, HumanEval) для масового тестування, а також A/B-тестування на реальних користувачах.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Під час етапу донавчання (instruction tuning) моделі бачили мільярди прикладів, де фрагменти коду чи JSON навмисно обгорталися в markdown-блоки для красивого читання людиною. Ця глибока звичка переважає негативну текстову інструкцію «без markdown».

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Обсяг українськомовних текстів у загальному масиві даних для попереднього навчання нейромережі становить лише малу частку порівняно з англійською мовою. Через це модель має гірше статистичне уявлення про граматику, синтаксис та специфічну термінологію української мови.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Це пов'язано з параметром температури (temperature > 0), який додає випадковість (стохастичність) у вибір наступного токена на основі ймовірнісного розподілу. У результаті шлях генерації тексту щоразу змінюється.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація — це впевнена генерація неправдивої інформації, яка виглядає правдоподібно. Оскільки архітектура LLM не звертається до зовнішньої реляційної бази даних, а лише передбачає найбільш статистично ймовірне продовження тексту, вона не має вбудованого механізму усвідомлення власного незнання і завжди видає текст за аналогією.

Власні короткі висновки для звіту:

Надійність рубрики: Оцінювання за фіксованою рубрикою є надзвичайно надійним інструментом в інженерії, оскільки перетворює суб'єктивні враження на чіткі об'єктивні бали.

Найпоказовіші завдання: Найбільш показовими виявилися завдання на формат (JSON) та логіку, оскільки саме вони чітко розділяють моделі за рівнем придатності для реальних програмних проєктів.

Вибір моделі: Для роботи з українськомовним текстом та інтеграції в програмну систему однозначно варто обирати хмарну модель (Gemini), оскільки вона володіє кращою стабільністю, значно вищою якістю мови та точніше дотримується форматування.

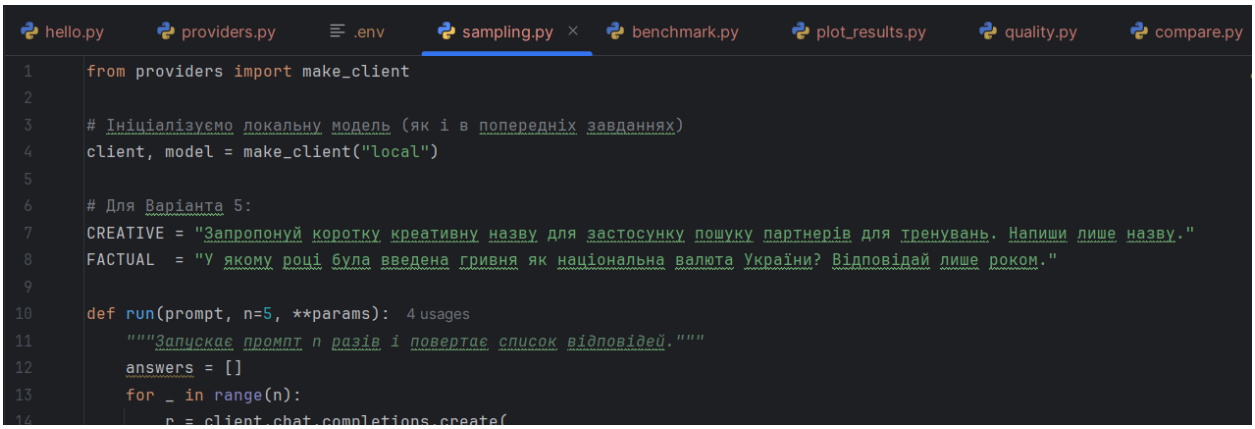
Крок 7. Дослідити вплив temperature і top_p

Завдання 6. Керування випадковістю: temperature і top_p

Мій варіант завдань:

№	***Творче завдання***	***Фактичне питання***
5	Назва застосунку пошуку партнерів для тренувань	Рік уведення гривні

Створити файл sampling.py:



```
(venv) C:\Users\Victus\lab1>python sampling.py
=== ДОСЛІДЖЕННЯ TEMPERATURE (Творче завдання) ===
temperature=0.0: унікальних 1 з 5
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"

temperature=0.7: унікальних 5 з 5
    "Тренувальний партнер"
    "ТренерТи"
    "Тренер на дому"
    "Тренер на зустрічі!"
    "Тренер Допомога"

temperature=1.5: унікальних 5 з 5
    "Тренери з іншими"
    FitPal
    FitMatch
    "Partner Fitness"
    'PartnerFit'

=== ДОСЛІДЖЕННЯ TOP_P (Творче завдання) ===
top_p=0.1: унікальних 1 з 5
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"
    "Тренер на дистанцію"

top_p=0.5: унікальних 5 з 5
    "Тренерський партнер"
    "Тренер на дистанцію"
    "Тренер на лайф"
    "Тренерські зустрічі"
    "Тренер на дому"

top_p=1.0: унікальних 5 з 5
    "Тренер на зустрічі"
    "Trenuj з нами!"
    FitPartner
    "Фіт Фенд"
    "Тренер на дистанцію"

=== ФАКТИЧНЕ ПИТАННЯ ПРИ різній TEMPERATURE ===
temperature=0.0: ['1992.', '1992.', '1992.', '1992.', '1992.']
temperature=1.5: ['1922.', '1961.', '1996.', '2022.', '1992.']
=== ПЕРЕВІРКА ВІДТВОРЮВАНОСТІ (10 запусків при t=0.0) ===
Усі відповіді однакові: True
Отримані відповіді: ['1992.', '1992.', '1992.', '1992.', '1992.', '1992.', '1992.', '1992.', '1992.', '1992.']
```

Вплив параметрів на різноманітність відповідей

Параметр	***Значення***	***Унікальних із 5 (творче)***	***Правильних із 5 (фактичне)***
temperature	0.0	1	0 (5/5 однакових невірних: 1992.)
temperature	0.7	5	—
temperature	1.5	5	1 (1996 — 1 раз із 5)
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Перевірка відтворюваності

Показник	***Значення***
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

temperature = 0.0, оскільки для генерації машинозчитуваних даних критично важлива максимальна детермінованість, чітке дотримання синтаксису та відсутність творчих відхилень.

2. Яке — генератору варіантів заголовків?

Поставлю високе значення (0.7 – 1.0), щоб «розплющити» розподіл ймовірностей та змусити модель видавати різноманітні, несподівані й креативні варіанти.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Temperature змінює форму всього розподілу ймовірностей (згладжує або загострює його), впливаючи на відносні шанси всіх токенів. Top_p діє як жорсткий динамічний фільтр («хвостове» відсікання), який відкидає малоймовірні токени за сукупною сумою їх імовірностей, не чіпаючи відносне співвідношення лідерів.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

У переважній більшості випадків так. Експеримент із 10 запусків показав повну відтворюваність — усі 10 відповідей виявилися абсолютно ідентичними (1992.).

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Приклад: Видача випадкових років на кшталт 1922 або 2022 на фактичне питання. Вона з'явилася через те, що висока температура підняла ваги малоймовірних токенів у хвості розподілу, зруйнувавши фактичну точність моделі.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Він ділить вихідні логарифми (logits) на значення температури перед застосуванням функції softmax, сплющуючи або роблячи пік розподілу гострішим.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Temperature змінює «крутизну» всього спектра ймовірностей, тоді як top_p сортує токени за спаданням і відсікає хвіст розподілу, сума ймовірностей якого перевищує заданий поріг.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Зазвичай рекомендується змінювати лише один із параметрів (найчастіше temperature), оскільки їх одночасна корекція ускладнює контроль над поведінкою виводу моделі.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Це зумовлено апаратними особливостями обчислень із плаваючою комою на різних прискорювачах та невеликою недетермінованістю в паралельних бібліотеках виконання.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для структурованих даних використовують 0.0 – 0.2, а для творчих завдань та генерації ідей — 0.7 – 1.0.

Власні висновки:

Для наступних лабораторних робіт, де модель має повертати структуровані дані, я обведу значення temperature = 0.0 та top_p = 1.0 (або за замовчуванням). Налаштування «за замовчуванням» (яке часто дорівнює 0.7 або 1.0) виявиться непридатним для програмних задач, оскільки випадковість виводу неминуче призведе до збоїв парсингу JSON та руйнування логіки агентної системи.

Крок 8. Звести все разом і зробити висновок

Підсумкове порівняння моделей

Критерій	***Локальна мала (3B)***	***Локальна більша (4B/8B)***	***Хмарна***
Швидкість генерації	Висока (на коротких текстах) / Середня	Середня	Висока (завдяки серверним прискорювачам)
TTFT	Низький	Низький / Середній	Вищий
Якість, балів /12	Низька (5/12)	Середня (7–9/12)	Висока (12/12)
Стабільність формату (завдання 2)	Низька	Середня	Висока
Потрібні ресурси	Низькі (достатньо звичайного ноутбука)	Середні (потребує дискретної GPU)	На пристрої відсутні (всі обчислення в хмарі)
Вартість	Безкоштовно	Безкоштовно	Умовно безкоштовно (обмеження безкоштовного тарифу / платне API)
Куди йдуть ваші дані	Залишаються на пристрої (повна конфіденційність)	Залишаються на пристрої	Передаються на сторонні серверні дата-центри
Працює без інтернету	Так	Так	Ні

Критерій	*Локальна мала (3B)*	*Локальна більша (4B/8B)*	*Хмарна*
Коли обирати саме її	Для простих текстових завдань, простих замінників коду або за жорсткої потреби повної конфіденційності й роботи офлайн.	Для складніших локальних задач, коли є трохи більше обчислювальних ресурсів пристрою.	Для складних агентних систем, роботи з точним JSON-форматом, багатокрокової логіки та українськомовного контенту високої якості.

Висновок:

Лабораторна робота показала, що вибір мовної моделі та налаштування її параметрів критично впливають на якість роботи AI-систем. Малі локальні моделі швидкі та забезпечують конфіденційність, але поступаються в логіці й точності форматування, тоді як хмарні рішення демонструють високу якість і стабільність на складних завданнях. Водночас параметр температури діє як регулятор випадковості: нульове значення забезпечує необхідну для програмного коду й структурованих даних відтворюваність, тоді як висока температура руйнує фактичну точність заради креативності. У результаті створення ефективних агентних систем вимагає зваженого балансу між вимогами до конфіденційності, швидкості, точності виводу та наявністю мережі.